

Existen algunos inconvenientes ocultos en las declaraciones y definiciones externas si se utilizan varios archivos fuente. Para evitarlos, primero defina e inicialice cada variable externa solo una vez en todo el conjunto de archivos:

```
entero    foo =    0;
```

Puedes usar múltiples definiciones externas en UNIX, pero no en GCOS, así que no lo hagas.

Buscar problemas. Las inicializaciones múltiples son ilegales en todas partes. Segundo, al principio de cualquier archivo que contenga funciones que necesiten una variable cuya definición esté en otro archivo, coloque una declaración extern, fuera de cualquier función:

```
externo    foo;

f1() { ... }           etc.
```

#definir, #incluir

C proporciona una funcionalidad de macros muy limitada. Se puede decir:

```
# definir nombre    algo
```

y a partir de entonces, en cualquier lugar donde aparezca "nombre" como token, se sustituirá por "algo". Este es particularmente útil para parametrizar los tamaños de los arreglos:

```
# definir TAMAÑO DE LA MATRIZ    100
entero    arr[TAMAÑO DE LA MATRIZ];
...
mientras (i++ < TAMAÑO DEL ARRAY)...
```

(ahora podemos alterar todo el programa cambiando solo la definición) o configurando constantes misteriosas:

```
# definir CONJUNTO    01
# define INTERRUPCIÓN    02    /* bit de interrupción */
# define HABILITADO 04
...
sí( x & (CONFIGURAR | INTERRUMPIR | HABILITADO) ) ...
```

Ahora tenemos palabras significativas en lugar de constantes misteriosas. (Los operadores misteriosos) &' (Y) y '|' (O) se tratarán en [la siguiente sección](#).) Es una excelente práctica escribir programas sin constantes literales excepto en las declaraciones #define.

Hay varias advertencias sobre #define. Primero, no hay punto y coma al final de un # define; todo el texto desde el nombre hasta el final de la línea (excepto los comentarios) se considera "algo". Cuando se inserta en el texto, se colocan espacios en blanco a su alrededor. El otro

La palabra de control conocida por C es `#include`. Para incluir un archivo en su código fuente en tiempo de compilación, diga:

```
# incluir "nombre de archivo"
```

Operadores de bits

C tiene varios operadores para operaciones lógicas de bits. Por ejemplo,

```
x = x & 0177;
```

forma la operación AND bit a bit de `x` y `0177`, conservando efectivamente solo los últimos siete bits de `x`. los operadores son

Otro

	<i>incluido O</i>	
^	<i>(circunflejo) exclusivo O (tilde)</i>	
~	<i>complemento a 1</i>	
!	<i>NO lógico</i>	
<<	<i>desplazamiento a la izquierda (como en x<<2)</i>	
>>	<i>desplazamiento a la derecha</i>	<i>(aritmética en PDP-11; lógico en H6070,</i>
	<i>IBM360)</i>	

Operadores de asignación

Una característica inusual de C es que los operadores binarios normales como `+`, `-`, etc. pueden ser combinado con el operador de asignación `=` para formar nuevos operadores de asignación. Por ejemplo,

```
x = -10;
```

utiliza el operador de asignación `-=` para decrementar `x` por 10, y

```
x =& 0177
```

forma el AND de `x` y `0177`. Esta convención es un atajo de notación útil, particularmente si `x` es una expresión compleja. El ejemplo clásico es la suma de un array:

```
para( suma=i=0; i<n; i++ )
    suma = + array[i];
```

Pero los espacios alrededor del operador son críticos.

```
x = -10;
```

también disminuye `x` en 10. Esto es bastante contrario a la experiencia de la mayoría de los programadores. En particular, tenga cuidado con cosas como

```
c=*s++;
y=&x[0];
```

Ambas opciones casi con toda seguridad no son lo que usted deseaba. Versiones más recientes de varios compiladores Tienen la cortesía de advertirle sobre la ambigüedad.

Dado que todos los demás operadores de una expresión se evalúan antes que el operador de asignación, se debe prestar mucha atención al orden de evaluación:

```
x = x<<y | z;
```

significa "desplazar x a la izquierda y posiciones, luego realizar una operación OR con z y almacenar en x".

Punto flotante

C tiene números de precisión simple y doble. Por ejemplo,

```
suma doble;
promedio flotante, y[10];
suma = 0.0;
para( i=0; i<n; i++ )
    suma += y[i]; promedio = suma/n;
```

Toda la aritmética de punto flotante se realiza en doble precisión. La aritmética de modo mixto es válida; si un operador aritmético en una expresión tiene ambos operandos `entero` o `carbonizarse`, la aritmética realizada es entera, pero si un operando es `entero` o `carbonizarse` el otro es `flotar` o `doble`, ambos operandos se convierten en `doble`. Por lo tanto, si `y` es `entero` y `i` es `incógnita` es `flotar`,

$\frac{(x+i)/j}{x + i/j}$	<i>converso si <code>y</code> es <code>flotar</code> hace <code>y</code> o <code>yo</code> <code>entero</code>, luego convierte</i>
---------------------------	---

La conversión de tipos se puede realizar mediante asignación; por ejemplo,

```
entero m, n;
x, y;
m = x;
y = n;
```

`converso` `incógnita` a `entero` (truncando hacia cero), y `no` a `punto flotante`.

Las constantes de punto flotante son iguales que las de Fortran o PL/I, excepto que la letra del exponente es 'e' en lugar de 'E'. Por lo tanto:

```
pi = 3,14159;
grande = 1,23456789e10;
```

imprimirf formateará números de punto flotante: "%w.df" en la cadena de formato imprimirá la variable correspondiente en un campo de w dígitos de ancho, con d posiciones decimales. Una e en lugar de una f producirá notación exponencial.

ir ay etiquetas

C tiene una instrucción goto y etiquetas, por lo que puedes ramificar como solías hacerlo. Pero la mayoría La mayoría de las veces no se necesitan goto. (¿Cuántos hemos usado hasta ahora?) El código casi siempre se puede expresar de forma más clara con for/while, if/else y sentencias compuestas.

Un uso de goto con cierta legitimidad es en un programa que contiene un bucle largo, donde un mientras(1) sería demasiado extenso. Entonces podrías escribir

```
bucle principal:
...
    ir al bucle principal;
```

Otro uso es implementar unromperde más de un nivel deparaomientras. Solo se permite goto's lata ramificar a etiquetas dentro de la misma función.

Manipulación de cadenas

Una cadena es una secuencia de caracteres, cuyo inicio está indicado por un puntero y cuyo final está marcado por el carácter nulo \0. En ocasiones, es necesario conocer la longitud de una cadena (el número de caracteres entre el inicio y el final de la cadena) [5]. Esta longitud se obtiene con la función de biblioteca strlen(). Su prototipo, en STRING.H, es

```
tamaño_t strlen(char *str);
```

La función strcpy()

La función de biblioteca strcpy() copia una cadena completa a otra ubicación de memoria. Su prototipo es el siguiente:

```
char *strcpy( char *destino, char *origen );
```

Antes de usar strcpy(), debe asignar espacio de almacenamiento para la cadena de destino.

```
/* Demuestra strcpy(). */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char source[] = "La cadena de origen.";
```

```
principal()
{
    carácter dest1[80];
    carácter *destino2, *destino3;

    printf("\nfuente: %s", fuente );

    /* Copiar a dest1 está bien porque dest1 apunta a */ /* 80 bytes de
    espacio asignado. */

    strcpy(dest1, source);
    printf("\ndest1: %s", dest1);

    /* Para copiar a dest2 debes asignar espacio. */ dest2 = (char
    *)malloc(strlen(source) + 1); strcpy(dest2, source);

    printf("\ndest2: %s\n", dest2);

    devolver(0);
}
origen: La cadena de origen.
destino1: La cadena de origen.
destino2: La cadena de origen.
```

La función strncpy()

La función strncpy() es similar a strcpy(), excepto que strncpy() permite especificar cuántos caracteres copiar. Su prototipo es

```
char *strncpy(char *destino, char *origen, size_t n);
```

```
/* Usando la función strncpy(). */

#include <stdio.h>
#include <string.h>

char dest[] = "....."; char source[] =
"abcdefghijklmnopqrstuvwxyz";

principal()
{
    tamaño_t n;

    mientras (1)
    {
        puts("Ingrese el número de caracteres a copiar (1-26)"); scanf("%d", &n);

        si (n > 0 && n < 27)
            romper;
    }
}
```

```
printf("\nAntes de strncpy destino = %s", dest);  
strncpy(destino, origen, n);  
printf("\nDespués de strncpy destino = %s\n", dest); return(0); }
```

Introduzca el número de caracteres a copiar (1-26) **15**

Antes de strncpy destino = Después de strncpy destino =
abcdefghijklmno.....

La función strdup()

La función de biblioteca strdup() es similar a strcpy(), excepto que strdup() realiza su propia asignación de memoria para la cadena de destino con una llamada a malloc(). El prototipo de strdup() es

```
char *strdup( char *source );
```

Utilizar strdup() para copiar una cadena con asignación automática de memoria.

```
/* La función strdup(). */  
#incluir <stdlib.h>  
#incluir <stdio.h>  
# incluir <string.h>  
  
char source[] = "La cadena de origen.";  
  
principal()  
{  
    carácter *destino;  
  
    si ( (destino = strdup(origen)) == NULL) {  
        fprintf(stderr, "Error al asignar memoria."); exit(1);  
    }  
  
    printf("El destino = %s\n", dest); return(0);  
}  
El destino = La cadena de origen.
```

La función strcat()

El prototipo de strcat() es

```
char *strcat(char *str1, char *str2);
```

La función agrega una copia de str2 al final de str1, moviendo el carácter nulo de terminación al final de la nueva cadena. Debes asignar suficiente espacio para que str1 almacene la cadena resultante. El valor de retorno de strcat() es un puntero a str1. El siguiente listado muestra el funcionamiento de strcat().

```
/* La función strcat(). */

#include <stdio.h>
#include <string.h>

char str1[27] = "a"; char
str2[2];

principal()
{
    entero n;

    /* Coloca un carácter nulo al final de str2[]. */

    str2[1] = '\0';

    para (n = 98; n < 123; n++) {

        str2[0] = n;
        strcat(str1, str2);
        puts(str1);
    }
    devolver(0);
}
```

```
ab
abecedario
abcd
abcde
abcdef
abcdefg
abcdefgh
abcdefghi
abcdefghij
abcdefghijk
abcdefghijkl
abcdefghijklm
abcdefghijklmn
abcdefghijklmno
```

```
abcdefghijklmnop  
abcdefghijklmnopq  
abcdefghijklmnopqr  
abcdefghijklmnopqrs  
abcdefghijklmnopqrst  
abcdefghijklmnopqrstu  
abcdefghijklmnopqrstuv  
abcdefghijklmnopqrstuvw  
abcdefghijklmnopqrstuvwx  
abcdefghijklmnopqrstuvwxy  
abcdefghijklmnopqrstuvwxyz
```

Comparación de cadenas

Se comparan cadenas de caracteres para determinar si son iguales o diferentes. Si son diferentes, una cadena es "mayor que" o "menor que" la otra. Estas determinaciones se realizan mediante los códigos ASCII de los caracteres. En el caso de las letras, esto equivale al orden alfabético, con la única excepción, aparentemente extraña, de que todas las letras mayúsculas son "menores que" las minúsculas. Esto se debe a que las letras mayúsculas tienen códigos ASCII del 65 al 90 (de la A a la Z), mientras que las minúsculas (de la a a la z) están representadas por los códigos del 97 al 122. Por lo tanto, "ZEBRA" se consideraría menor que "apple" según estas funciones de C.

La biblioteca ANSI C contiene funciones para dos tipos de comparaciones de cadenas: comparar dos cadenas completas y comparar un número determinado de caracteres en dos cadenas.

Comparación de dos cadenas completas

La función `strcmp()` compara dos cadenas carácter por carácter. Su prototipo es

```
int strcmp(char *str1, char *str2);
```

Los argumentos `str1` y `str2` son punteros a las cadenas que se comparan. Los valores de retorno de la función se muestran en la tabla. El siguiente listado muestra el funcionamiento de `strcmp()`.

Los valores devueltos por `strcmp()`.

Valor de retorno	Significado
< 0	str1 es menor que str2.
0	str1 es igual a str2.
> 0	str1 es mayor que str2.

Utilizar strcmp() para comparar cadenas.

```
/* La función strcmp(). */

#include <stdio.h>
#include <string.h>

principal()
{
    char str1[80], str2[80]; int x;

    mientras (1)
    {
        /* Introduce dos cadenas de texto. */
        printf("\n\nIngrese la primera cadena, deje un espacio en blanco para salir:");
        gets(str1);

        si ( strlen(str1) == 0 )
            romper;

        printf("\nIngrese la segunda cadena: "); gets(str2);

        /* Compáralos y muestra el resultado. */
        x = strcmp(str1, str2);

        printf("\nstrcmp(%s,%s) devuelve %d", str1, str2, x);
    }
    devolver(0);
}
```

Introduzca la primera cadena; deje un espacio en blanco para salir:**Primera línea**
Introduzca la segunda cadena:**Segunda línea** strcmp(Primera cadena,Segunda
cadena) devuelve -1. Introduzca la primera cadena; deje un espacio en blanco
para salir:**cadena de prueba** Introduzca la segunda cadena:**cadena de prueba**
strcmp(cadena de prueba,cadena de prueba) devuelve 0. Introduzca la primera
cadena; un espacio en blanco para salir:**cebra** Introduzca la segunda cadena:
cerdo hormiguero strcmp(zebra,"aardvark") devuelve 1. Introduzca la primera
cadena; un espacio en blanco para salir:

Comparación de cadenas parciales

La función de biblioteca strncmp() compara un número específico de caracteres de una cadena con otra cadena. Su prototipo es

```
int strncmp(char *str1, char *str2, size_t n);
```

La función `strncmp()` compara `n` caracteres de `str2` con `str1`. La comparación continúa hasta que se hayan comparado `n` caracteres o se haya llegado al final de `str1`. El método de comparación y los valores de retorno son los mismos que para `strcmp()`. La comparación distingue entre mayúsculas y minúsculas.

Comparación de partes de cadenas con `strncmp()`.

```
/* La función strncmp(). */

#include <stdio.h>
#include <string.h>

char str1[] = "La primera cadena."; char str2[] =
"La segunda cadena."; main()

{
    tamaño_t n, x;

    puts(str1);
    puts(str2);

    mientras (1)
    {
        puts("\n\nIngrese el número de caracteres a comparar, 0 para salir."); scanf("%d",
        &n);

        si (n <= 0)
            romper;

        x = strncmp(str1, str2, n);

        printf("\nComparando %d caracteres, strncmp() devuelve %d.", n, x);
    }
    devolver(0);
}
```

La primera cuerda.

La segunda cuerda.

Introduzca el número de caracteres a comparar; 0 para salir. **3**

Comparando 3 caracteres, `strncmp()` devuelve .©] Introduzca el
número de caracteres a comparar, 0 para salir. **6**

Al comparar 6 caracteres, `strncmp()` devuelve -1. Introduzca el
número de caracteres a comparar; 0 para salir. **0**

La función strchr()

La función strchr() encuentra la primera aparición de un carácter específico en una cadena. El prototipo es

```
char *strchr(char *str, int ch);
```

La función strchr() busca en str de izquierda a derecha hasta encontrar el carácter ch o el carácter nulo de terminación. Si se encuentra ch, se devuelve un puntero a él. Si no, se devuelve NULL.

Cuando strchr() encuentra un carácter, devuelve un puntero a ese carácter. Sabiendo que str es un puntero al primer carácter de la cadena, se puede obtener la posición del carácter encontrado restando str del valor del puntero devuelto por strchr(). El siguiente listado lo ilustra. Recuerde que el primer carácter de una cadena se encuentra en la posición 0. Al igual que muchas funciones de cadena de C, strchr() distingue entre mayúsculas y minúsculas. Por ejemplo, indicaría que el carácter 'F' no se encuentra en la cadena 'raffle'.

Utilizar strchr() para buscar un solo carácter en una cadena de texto.

```
/* Búsqueda de un solo carácter con strchr(). */

#include <stdio.h>
#include <string.h>

principal()
{
    char *loc, buf[80];
    canal de int;

    /* Introduce la cadena y el carácter. */

    printf("Ingrese la cadena a buscar: "); gets(buf);
    printf("Ingrese el carácter a buscar: "); ch = getchar();

    /* Realizar la búsqueda. */

    loc = strchr(buf, ch);

    si (loc == NULL)
        printf("El carácter %c no fue encontrado.", ch);
    demás
        printf("El carácter %c se encontró en la posición %d.\n",
               ch, loc-buf);

    return(0); }
```

Introduzca la cadena de texto que desea buscar: **¿Qué tal, Vaca Marrón?** Introduzca el personaje que desea buscar: **do** El carácter C se encontró en la posición 14.

La función strcspn()

La función de biblioteca strcspn() busca en una cadena la primera aparición de cualquiera de los caracteres de una segunda cadena. Su prototipo es

```
tamaño_t strcspn(char *str1, char *str2);
```

La función strcspn() comienza la búsqueda en el primer carácter de str1, buscando cualquiera de los caracteres individuales contenidos en str2. Es importante recordar esto. La función no busca la cadena str2, sino solo los caracteres que contiene. Si la función encuentra una coincidencia, devuelve el desplazamiento desde el principio de str1, donde se encuentra el carácter coincidente. Si no encuentra ninguna coincidencia, strcspn() devuelve el valor de strlen(str1). Esto indica que la primera coincidencia fue el carácter nulo que termina la cadena.

Búsqueda de un conjunto de caracteres con strcspn().

```
/* Búsqueda con strcspn(). */

#include <stdio.h>
#include <string.h>

principal()
{
    carbonizarse    buf1[80], buf2[80];
    tamaño_t loc;

    /* Introduce las cadenas. */

    printf("Ingrese la cadena a buscar: "); gets(buf1);
    printf("Ingrese la cadena que contiene los caracteres objetivo: "); gets(buf2);

    /* Realizar la búsqueda. */

    loc = strcspn(buf1, buf2);

    si ( loc == strlen(buf1) )
        printf("No se encontró ninguna coincidencia.");
    demás
        printf("Se encontró la primera coincidencia en la posición %d.\n", loc); return(0);
}
```

Introduzca la cadena de texto que desea buscar: **¿Qué tal, Vaca Marrón?**
Introduzca la cadena que contiene los caracteres deseados: **Gato** La primera coincidencia se encontró en la posición 14.

La función strpbrk()

La función de biblioteca strpbrk() es similar a strcspn(), ya que busca en una cadena la primera aparición de cualquier carácter contenido en otra cadena. Se diferencia en que no incluye los caracteres nulos de terminación en la búsqueda. El prototipo de la función es

```
char *strpbrk(char *str1, char *str2);
```

La función strpbrk() devuelve un puntero al primer carácter de str1 que coincide con alguno de los caracteres de str2. Si no encuentra ninguna coincidencia, la función devuelve NULL. Como se explicó anteriormente para la función strchr(), se puede obtener el desplazamiento de la primera coincidencia en str1 restando el puntero str1 del puntero devuelto por strpbrk() (siempre que no sea NULL, por supuesto).

La función strstr()

La última función de búsqueda de cadenas en C, y quizás la más útil, es strstr(). Esta función busca la primera aparición de una cadena dentro de otra, y busca en toda la cadena, no en caracteres individuales dentro de ella. Su prototipo es

```
char *strstr(char *str1, char *str2);
```

La función strstr() devuelve un puntero a la primera aparición de str2 dentro de str1. Si no encuentra ninguna coincidencia, devuelve NULL. Si la longitud de str2 es 0, devuelve str1. Cuando strstr() encuentra una coincidencia, se puede obtener la posición de str2 dentro de str1 mediante la resta de punteros, como se explicó anteriormente para strchr(). El procedimiento de coincidencia que utiliza strstr() distingue entre mayúsculas y minúsculas.

Utilizar strstr() para buscar una cadena dentro de otra.

```
/* Búsqueda con strstr(). */

#include <stdio.h>
#include <string.h>

principal()
{
    char *loc, buf1[80], buf2[80];

    /* Introduce las cadenas. */

    printf("Ingrese la cadena que desea buscar: ");
```

```
obtiene(buf1);
printf("Ingrese la cadena de destino: ");
gets(buf2);

/* Realizar la búsqueda. */

loc = strstr(buf1, buf2);

si (loc == NULL)
    printf("No se encontró ninguna coincidencia.\n");
demás
    printf("%s se encontró en la posición %d.\n", buf2, loc-buf1); return(0);}
```

Introduzca la cadena de texto que desea buscar: **¿Qué tal, vaca marrón?**
Introduzca la cadena de destino: **vaca** La vaca fue encontrada en la posición 14.

La función strrev()

La función strrev() invierte el orden de todos los caracteres de una cadena (no es estándar ANSI). Su prototipo es

```
char *strrev(char *str);
```

El orden de todos los caracteres en str se invierte, y el carácter nulo de terminación permanece al final.

Conversiones de cadena a número

En ocasiones, necesitará convertir la representación en cadena de texto de un número a una variable numérica. Por ejemplo, la cadena "123" se puede convertir a una variable de tipo entero con el valor 123. Existen tres funciones para convertir una cadena a un número. Estas se explican en las siguientes secciones; sus prototipos se encuentran en STDLIB.H.

La función atoi()

La función de biblioteca atoi() convierte una cadena en un número entero. El prototipo es

```
int atoi(char *ptr);
```

La función atoi() convierte la cadena a la que apunta ptr en un número entero. Además de dígitos, la cadena puede contener espacios en blanco al principio y un signo + o -. La conversión comienza desde el principio.

La función recorre la cadena y continúa hasta encontrar un carácter no convertible (por ejemplo, una letra o un signo de puntuación). El entero resultante se devuelve al programa que la llamó. Si no encuentra caracteres convertibles, `atoi()` devuelve 0. La tabla muestra algunos ejemplos.

Conversiones de cadena a número con `atoi()`.

Cadena	Valor devuelto por <code>atoi()</code>
"157"	157
"-1.6"	- 1
"+50x"	50
"doce"	0
"x506"	0

La función `atol()`

La función de la biblioteca `atol()` funciona exactamente igual que `atoi()`, excepto que devuelve un tipo `long`. El prototipo de la función es

```
long atol(carácter *ptr);
```

La función `atof()`

La función `atof()` convierte una cadena a un tipo `double`. El prototipo es

```
double atof(char *str);
```

El argumento ``str`` apunta a la cadena que se va a convertir. Esta cadena puede contener espacios en blanco al principio y un carácter `+` o `-`. El número puede contener los dígitos del 0 al 9, el punto decimal y el exponente `E` o `e`. Si no hay caracteres convertibles, ``atof()`` devuelve 0. La tabla 17.3 muestra algunos ejemplos de uso de ``atof()``.

Conversiones de cadena a número con `atof()`.

Cadena	Valor devuelto por <code>atof()</code>
"12"	12.000000
"-0.123"	- 0,123000
"123E+3"	123000.000000
"123.1e-5"	0,001231

Funciones de prueba de carácter

El archivo de cabecera CTYPE.H contiene los prototipos de varias funciones que prueban caracteres, devolviendo VERDADERO o FALSO dependiendo de si el carácter cumple una determinada condición. Por ejemplo, ¿es una letra o un número?xxxxLas funciones () son en realidad macros, definidas en CTYPE.H.

El esxxxxTodas las macros () tienen el mismo prototipo:

```
entero esxxxx(int ch);
```

En la línea anterior, ch es el carácter que se está probando. El valor de retorno es VERDADERO (distinto de cero) si se cumple la condición o FALSO (cero) si no se cumple. La tabla enumera el conjunto completo de esxxxx() macros.

Las macros isxxxx().

Macro	Acción
isalnum()	Devuelve VERDADERO si ch es una letra o un dígito.
isalpha()	Devuelve VERDADERO si ch es una letra.
isascii()	Devuelve VERDADERO si ch es un carácter ASCII estándar (entre 0 y 127).
iscntrl()	Devuelve VERDADERO si ch es un carácter de control.
esdígito()	Devuelve VERDADERO si ch es un dígito.
esgrafo()	Devuelve VERDADERO si ch es un carácter imprimible (distinto de un espacio).
esinferior()	Devuelve VERDADERO si ch es una letra minúscula.
isprint()	Devuelve VERDADERO si ch es un carácter imprimible (incluido un espacio).
ispunct()	Devuelve VERDADERO si ch es un carácter de puntuación.
esespacio()	Devuelve VERDADERO si ch es un carácter de espacio en blanco (espacio, tabulación, tabulación vertical, salto de línea, salto de página o retorno de carro).
isupper()	Devuelve VERDADERO si ch es una letra mayúscula.
isxdigit()	Devuelve VERDADERO si ch es un dígito hexadecimal (del 0 al 9, de la a a la f, de la A a la F).

Puedes hacer muchas cosas interesantes con las macros de prueba de caracteres. Un ejemplo es la función `get_int()`, que se muestra en el listado. Esta función recibe un número entero desde la entrada estándar y lo devuelve como una variable de tipo `int`. La función omite los espacios en blanco iniciales y devuelve 0 si el primer carácter que no es un espacio no es un carácter numérico.

Utilizar las macros isxxxx() para implementar una función que reciba como entrada un número entero.

```
/* Usando macros de prueba de caracteres para crear una función de
entrada entera. */ /*
#include <stdio.h>
```



```
# incluir <ctype.h>

entero obtener_int(void);

principal()
{
    entero x;
    x = obtener_int();
    printf("Has introducido %d.\n", x);
}

entero obtener_int(void)
{
    entero ch, i, signo = 1;

    mientras ( esespacio(ch = obtenerchar()));

    si (ch != '-' && ch != '+' && !isdigit(ch) && ch != EOF) {
        ungetc(ch, stdin);
        devolver 0;
    }

    /* Si el primer carácter es un signo menos, establezca */ /* el
    signo en consecuencia. */

    si (ch == '-')
        signo = -1;

    /* Si el primer carácter fue un signo más o menos, */ /* obtener el
    siguiente carácter. */

    si (ch == '+' || ch == '-')
        ch = getchar();

    /* Lee caracteres hasta que se introduzca un carácter que no sea un dígito.
    Asigna */ /* valores, multiplicados por la potencia de 10 adecuada, a i. */

    para (i = 0; esdígito(ch); ch = obtenercarácter() )
        i = 10 * i + (ch - '0');

    /* Si el signo es negativo, el resultado será negativo. */

    yo *= signo;

    /* Si no se encontró EOF, se debe haber leído un carácter que no sea un
    dígito */ /* así que lo eliminamos. */
    si (ch != EOF)
        ungetc(ch, stdin);

    /* Devuelve el valor de entrada. */

    devolver i;
}
```

- 100
 Ingresaste -100.
abc3.145
 Ingresaste 0.
9 9 9
 Ingresaste 9.
2.5
 Ingresaste 2.

Funciones matemáticas

La biblioteca estándar de C contiene diversas funciones que realizan operaciones matemáticas. Los prototipos de estas funciones se encuentran en el archivo de cabecera MATH.H. Todas las funciones matemáticas devuelven un valor de tipo double. En el caso de las funciones trigonométricas, los ángulos se expresan en radianes. Recuerda que un radián equivale a 57,296 grados, y un círculo completo (360 grados) contiene 2π radianes.

Funciones trigonométricas

Función	Prototipo	Descripción
acos()	doble acos(doble x)	Devuelve el arcocoseno de su argumento. El argumento debe estar en el rango $-1 \leq x \leq 1$, y el valor de retorno está en el rango $0 \leq \text{acos} \leq \pi$.
asin()	doble asin(doble x)	Devuelve el arcoseno de su argumento. El argumento debe estar en el rango $-1 \leq x \leq 1$, y el valor de retorno está en el rango $-\pi/2 \leq \text{asin} \leq \pi/2$.
tan()	doble atan(doble x)	Devuelve la arcotangente de su argumento. El valor devuelto está en el rango $-\pi/2 \leq \text{atan} \leq \pi/2$.
atan2()	doble atan2(doble x, doble y)	Devuelve la arcotangente de x/y . El valor devuelto está en el rango $-\pi \leq \text{atan2} \leq \pi$.
cos()	doble cos(doble x)	Devuelve el coseno de su argumento.
sin()	doble seno(doble x)	Devuelve el seno de su argumento.
tan()	doble tan(doble x)	Devuelve la tangente de su argumento.

Funciones exponenciales y logarítmicas

Función	Prototipo	Descripción
exp()	doble exp(doble x)	Devuelve el exponente natural de su argumento, es decir, e^x donde e es igual a 2,7182818284590452354.
log()	doble log(doble x)	Devuelve el logaritmo natural de su argumento. El argumento debe ser mayor que 0.
log10()	doble logaritmo10(doble x)	Devuelve el logaritmo en base 10 de su argumento. El argumento debe ser mayor que 0.

Funciones hiperbólicas

Función	Prototipo	Descripción
aporrrear()	doble cosh(doble x)	Devuelve el coseno hiperbólico de su argumento.
sinh()	doble seno(doble x)	Devuelve el seno hiperbólico de su argumento.
tanh()	tanh doble(x doble)	Devuelve la tangente hiperbólica de su argumento.

Otras funciones matemáticas

Función	Prototipo	Descripción
raíz cuadrada()	doble raíz cuadrada (doble) incógnita)	Devuelve la raíz cuadrada de su argumento. El argumento debe ser cero o mayor.
fortificar techo()	techo doble (doble incógnita)	Devuelve el entero más pequeño no menor que su argumento. Por ejemplo, ceil(4.5) devuelve 5.0, y ceil(-4.5) devuelve -4.0. Aunque ceil() devuelve un valor entero, se devuelve como un número de tipo double.
abs()	entero abs(int x)	Devuelve el valor absoluto
laboratorios()	laboratorios largos(long x)	valor de sus argumentos.
piso()	piso doble (doble incógnita)	Devuelve el entero más grande que no sea mayor que su argumento. Por ejemplo, floor(4.5) devuelve 4.0 y floor(-4.5) devuelve -5.0.
modf()	doble modf(doble x, doble *y)	Divide x en partes enteras y fraccionarias, cada una con el mismo signo que x. La función devuelve la parte fraccionaria y asigna la parte entera a *y.
potencia()	doble potencia (doble x, doble y)	Devuelve x^y . Se produce un error si $x == 0$ e $y <= 0$, o si $x < 0$ y no es un número entero.

CAPÍTULO 4 ESTRUCTURAS DE DATOS ESENCIALES

En todo algoritmo, es necesario almacenar datos. Esto abarca desde almacenar un único valor en una sola variable hasta estructuras de datos más complejas. En los concursos de programación, hay varios aspectos de las estructuras de datos que se deben considerar al seleccionar la forma adecuada de representar los datos para un problema. Este capítulo le brindará algunas pautas y enumerará algunos aspectos básicos.

Estructuras de datos para empezar.[2]

¿Funcionará?

Si las estructuras de datos no funcionan, no sirven de nada. Pregúntese qué preguntas deberá formular el algoritmo a la estructura de datos y asegúrese de que esta pueda procesarlas. De lo contrario, deberá añadir más datos a la estructura o buscar una representación diferente.

¿Puedo programarlo?

Si no sabes o no recuerdas cómo codificar una estructura de datos determinada, elige otra. Asegúrate de comprender bien cómo afectará cada operación a la estructura de los datos.

Otro aspecto a considerar es la memoria. ¿Cabrán la estructura de datos en la memoria disponible? Si no, comprímala o elija una nueva. De lo contrario, queda claro desde el principio que no funcionará.

¿Podré programarlo a tiempo? ¿O mi lenguaje de programación aún no lo soporta?

Como se trata de un concurso con límite de tiempo, tienes que escribir entre tres y cinco programas en, digamos, cinco horas. Si tardas una hora y media en programar solo la estructura de datos para el primer problema, es casi seguro que estás usando la estructura equivocada.

Otra consideración muy importante es si tu lenguaje de programación te proporciona la estructura de datos necesaria. Para los programadores de C++, la STL ofrece una amplia gama de estructuras de datos integradas de gran calidad; lo que necesitas hacer es dominarlas, en lugar de intentar crear tu propia estructura de datos.

En la fase de competición, esta será una de las estrategias ganadoras. Consideremos este escenario. A todos los equipos se les da un conjunto de problemas. Algunos de ellos requieren el uso de 'stack'. El mejor programador del equipo A implementa directamente la mejor implementación de stack conocida; necesita 10 minutos para hacerlo. Sorprendentemente para ellos, los demás equipos escriben solo dos líneas:

"#include <stack>" y "std::stack<int> s;", ¿puedes ver quién tiene ahora 10 minutos de ventaja?

¿Puedo depurarlo?

Es fácil olvidar este aspecto particular de la selección de estructuras de datos. Recuerda que un programa es inútil si no funciona. Ten presente que el tiempo de depuración representa una parte importante del tiempo total del concurso, así que inclúyelo en el cálculo del tiempo de codificación.

¿Qué hace que una estructura de datos sea fácil de depurar? Eso viene determinado básicamente por las dos propiedades siguientes.

1. El estado es fácil de examinar Cuanto más pequeña y compacta sea la representación, en general, más fácil será examinarla. Además, los arreglos asignados estáticamente son...**mucho** Más fáciles de examinar que las listas enlazadas o incluso las matrices asignadas dinámicamente.

2. El estado se puede mostrar fácilmente Para las estructuras de datos más complejas, la forma más sencilla de examinarlas es escribir una pequeña rutina para generar la salida de datos. Desafortunadamente, debido a las limitaciones de tiempo, probablemente querrá limitarse a la salida de texto. Esto significa que estructuras como árboles y grafos serán difíciles de examinar.

¿Es rápido?

El uso de estructuras de datos más sofisticadas reducirá la complejidad general del algoritmo. Será conveniente que conozcas algunas estructuras de datos avanzadas.

Cosas que se deben evitar: Memoria dinámica

En general, conviene evitar la memoria dinámica, porque:

1. Es demasiado fácil cometer errores al usar memoria dinámica. Sobrescribir memoria previamente asignada, no liberar memoria y no asignar memoria son solo algunos de los errores que se producen al usar memoria dinámica. Además, los modos de fallo de estos errores son tales que resulta difícil determinar dónde se produjo el error, ya que probablemente ocurra en una operación de memoria (posiblemente mucho más adelante).

2. Es demasiado difícil examinar el contenido de la estructura de datos. Los entornos de desarrollo interactivos disponibles no manejan bien la memoria dinámica, especialmente para C. Considere los arreglos paralelos como una alternativa a la memoria dinámica. Una forma de hacer una lista enlazada, donde en lugar de mantener un punto siguiente, se mantiene un segundo arreglo, que tiene el índice del siguiente.

elemento. A veces puede que tengas que asignarlos dinámicamente, pero como solo debe hacerse una vez, es mucho más fácil hacerlo bien que asignar y liberar la memoria para cada inserción y eliminación.

A pesar de todo esto, a veces la memoria dinámica es la mejor opción, especialmente para estructuras de datos grandes donde el tamaño de la estructura no se conoce hasta que se recibe la entrada.

Cosas que evitar: Factor de frialdad

Intenta no caer en la trampa de lo "guay". Puede que acabes de ver la estructura de datos más elegante, pero recuerda:

1. Las ideas geniales que no funcionan no son.
2. Las ideas geniales que tardarán una eternidad en programarse tampoco lo son.

Es mucho más importante que la estructura de datos y el programa funcionen correctamente que lo impresionante que sea la estructura de datos.

Estructuras de datos básicas

Existen cinco estructuras de datos básicas: arreglos, listas enlazadas, pilas, colas y deque (una cola de doble extremo). No se tratarán en esta sección; consulte sus secciones correspondientes.

Matrices

Los arreglos son las estructuras de datos más útiles; de hecho, casi siempre se utilizan en todos los problemas de concursos. Veamos primero las ventajas: si se conoce el índice, buscar un elemento en un arreglo es muy rápido, $O(1)$, lo cual es bueno para bucles/iteraciones. Los arreglos también se pueden usar para implementar otras estructuras de datos sofisticadas como pilas, colas y tablas hash. Sin embargo, ser la estructura de datos más sencilla no significa que sea eficiente. En algunos casos, un arreglo puede ser muy ineficiente. Además, en un arreglo estándar, el tamaño es fijo. Si no se conoce el tamaño de entrada de antemano, puede ser más conveniente usar un vector (un arreglo redimensionable). Los arreglos también presentan una inserción muy lenta en arreglos ordenados, una búsqueda lenta en arreglos no ordenados y una eliminación inevitablemente lenta porque hay que desplazar todos los elementos.

Buscar	$O(n/2)$ comparaciones	$O(n)$ comparaciones
Inserción	Sin comparación, $O(1)$	Sin comparaciones, $O(1)$
Supresión	$O(n/2)$ comparaciones, $O(n/2)$ movimientos	$O(n)$ comparaciones más de $O(n/2)$ movimientos

Generalmente usamos matrices unidimensionales para uso estándar y matrices bidimensionales para representar matrices, tableros o cualquier cosa bidimensional. Las matrices tridimensionales rara vez se usan para modelar situaciones 3D.

mayor de remo, compatible con la caché, utilice este método para acceder a todos los elementos de la matriz de forma secuencial.

```
para (i=0; i<numRow; i++)          // FILA PRIMERO = EFECTIVO
    para (j=0; j<numCol; j++)
        array[i][j] = 0;
```

Columna mayor, NO compatible con caché, NO Utiliza este método o el rendimiento será muy bajo. ¿Por qué? Lo aprenderás en el curso de Arquitectura de Computadoras. Por ahora, ten en cuenta que las computadoras acceden a los datos de los arreglos por filas.

```
para (j=0; j<numCol; j++)          // COLUMNA PRIMERO = INEFICAZ
    para (i=0; i<numRow; i++)
        array[i][j] = 0;
```

Truco vectorial: una matriz redimensionable

Los arreglos estáticos presentan un inconveniente cuando se desconoce el tamaño de los datos de entrada. En ese caso, conviene considerar el uso de vectores, que son arreglos redimensionables.

Existen otras estructuras de datos que ofrecen una capacidad de redimensionamiento mucho más eficiente, como las listas enlazadas, etc.

CONSEJOS: UTILIZACIÓN DE LA STL VECTORIAL DE C++

La biblioteca estándar de C++ (STL) tiene plantillas vectoriales para ti.

```
#incluir <stdio.h>
# incluir <vector>
# incluir <algoritmo>
```

usando el espacio de nombres std;

```
void main() {
    // Simplemente haz esto, escribe vector<el tipo que desees, // en
    este caso, entero> y el nombre del vector vector<int>v;
```

```
    // intenta insertar 7 enteros diferentes, no ordenados v.retroceso
    (3); v.retroceso(1); v.retroceso(2);
    v.retroceso(7); v.retroceso(6); v.retroceso(5);
    v.retroceso(4);
```

```
// Para acceder al elemento, necesitas un iterador... vector<int>::
iterador;

printf("Versión sin ordenar\n");
// comienza con 'begin', termina con 'end', avanza con i++ para (i = v.
comenzar(); it= v.fin();yo++)
    printf("%d ",*i); // el puntero del iterador contiene el valor printf("\n");

clasificar(v.comenzar(),v.fin()); // ordenación predeterminada, ascendente

printf("Versión ordenada\n");
para (i = v.comenzar(); it= v.fin();yo++)
    printf("%d ",*i); // el puntero del iterador contiene el valor printf("\n");
}
```

Lista enlazada

Motivación para usar listas enlazadas: Un array es estático y, aunque tiene un tiempo de acceso $O(1)$ si se conoce el índice, debe desplazar sus elementos si se va a insertar o eliminar uno, lo cual es totalmente ineficiente. Un array no se puede redimensionar si está lleno (véase el ejemplo de array redimensionable: vector para una solución, pero el redimensionamiento es lento).

Las listas enlazadas permiten una inserción y eliminación muy rápidas. La ubicación física de los datos en una lista enlazada puede estar en cualquier parte de la memoria, pero cada nodo debe saber qué parte de la memoria corresponde al siguiente elemento.

Una lista enlazada puede ser tan grande como se desee, siempre que haya suficiente memoria. Un inconveniente de las listas enlazadas es que se desperdicia memoria cuando un nodo se "elimina" (solo se marca como eliminado). Esta memoria desperdiciada solo se libera cuando el recolector de basura realiza su función (depende del compilador y del sistema operativo utilizado).

Las listas enlazadas son estructuras de datos de uso común debido a su dinamismo. Sin embargo, no son fáciles de usar, ya que son muy complejas y tienden a generar errores de acceso a memoria en tiempo de ejecución. Algunos lenguajes de programación, como Java y C++, admiten la implementación de listas enlazadas mediante API (Interfaz de Programación de Aplicaciones) y STL (Biblioteca de Plantillas Estándar).

Una lista enlazada se compone de un dato (y a veces un puntero a ese dato) y un puntero al siguiente elemento. En una lista enlazada, solo se puede encontrar un elemento mediante una búsqueda exhaustiva desde el inicio hasta encontrarlo o hasta el final (si no se encuentra). Esta es la desventaja de las listas enlazadas, especialmente para listas muy largas. Para insertar un elemento en una lista enlazada ordenada, es necesario buscar en la posición adecuada mediante el método de búsqueda exhaustiva, lo cual también es lento. (Existen algunos trucos para mejorar la búsqueda en listas enlazadas, como recordar referencias a nodos específicos, etc.).

Variaciones de la lista enlazada

Con puntero de cola

En lugar del puntero de cabeza estándar, usamos otro puntero para llevar un registro del último elemento. Esto es útil para estructuras tipo cola, ya que en una cola, entramos desde el final (cola) y eliminamos los elementos desde el principio (cabeza).

Con nodos ficticios (centinelas)

Esta variación sirve para simplificar nuestro código (yo prefiero esta forma), puede simplificar el código de listas vacías y el código de inserción al principio de la lista.

Lista doblemente enlazada

Resulta eficiente si necesitamos recorrer la lista en ambas direcciones (hacia adelante y hacia atrás). Cada nodo ahora tiene dos punteros: uno apunta al siguiente elemento y el otro al anterior. Necesitamos un nodo inicial y un nodo final ficticios para este tipo de lista enlazada.

CONSEJOS: UTILIZACIÓN DE LA STL DE LISTAS DE C++

Una demostración sobre el uso de listas STL. La estructura de datos subyacente es una lista doblemente enlazada.

```
#incluir <stdio.h>

// Aquí es donde reside la implementación de la lista
# incluir <lista>

// utilice esto para evitar especificar "std::" en todas partes usando el
espacio de nombres std;

// Simplemente haz esto, escribe lista<el tipo que desees, // en
este caso, entero> y el nombre de la lista lista<int> l;

lista<int>::iteradori;

void print() {
    para (i = l.comenzar(); i != l.fin();yo++)
        printf("%d ",*i); // recuerda... ¡usa un puntero! printf("\n");
}

void main() {
    // Intenta insertar 8 enteros diferentes, tiene duplicados l.retroceso(3);
    l.retroceso(1); l.retroceso(2);
    l.retroceso(7); l.retroceso(6); l.retroceso(5);
    l.retroceso(4); l.retroceso(7); imprimir();
}
```

```
l.clasificar(); // ordenar la lista, wow ordenando lista enlazada... print();

l.eliminar(3); // elimina el elemento '3' de la lista print();

l.único(); // ¡Eliminar duplicados en la lista ORDENADA! print();

i = l.comenzar(); // establece el iterador al principio de la lista yo++
; // segundo nodo de la lista
l.insertar(i,1,10); // inserta aquí 1 copia de '10' print();
}
```

Pila

Estructuras de datos que solo permiten la inserción (**empujar**) y eliminación (**estallido**) solo desde arriba. Este comportamiento se llama Último en entrar, primero en salir (LIFO), similar a una pila normal en el mundo real.

Operaciones de pila importantes

1. Push (C++ STL: push())

Agrega un nuevo elemento en la parte superior de la pila.

2. Pop (C++ STL: pop())

Recupera y elimina la parte superior de una pila.

3. Mirar (C++ STL: top())

Recupera la parte superior de una pila sin eliminarla.

4. IsEmpty (C++ STL: empty())

Determina si una pila está vacía.

Algunas aplicaciones de pila

1. Para modelar una "pila real" en el mundo de la informática: recursión, llamada a procedimientos, etc.
2. Comprobar si un palíndromo es correcto (aunque comprobar si un palíndromo utiliza Queue & Stack es una tontería).
3. Para leer una entrada del teclado en la edición de texto con la tecla de retroceso.
4. Para invertir los datos de entrada (otra idea estúpida usar una pila para invertir datos).
5. Comprobar que los paréntesis estén equilibrados.
6. Cálculo del sufijo.
7. Conversión de expresiones matemáticas. Prefijo, infijo o postfijo.

Algunas implementaciones de pila

1. Lista enlazada con puntero solo al inicio (la mejor opción)
2. Matriz
3. Matriz redimensionable

CONSEJOS: UTILIZACIÓN DE LA STL DE LA PILA DE C++

Implementar una pila no es difícil. La implementación de Stack en la STL es muy eficiente, aunque será ligeramente más lenta que una pila personalizada.

```
#include <stdio.h>
# incluir <pila>

usando el espacio de nombres std;

void main() {
    // Simplemente haz esto, escribe stack<el tipo que deseas, // en
    // este caso, integer> y el nombre de la pila. pila<int>s;

    // Intenta insertar 7 números enteros diferentes, no ordenados.
    empujar(3); s.empujar(1); s.empujar(2);
    s.empujar(7); s.empujar(6); s.empujar(5);
    s.empujar(4);

    // El artículo que se inserta primero saldrá último // Orden Último en
    // entrar, primero en salir (LIFO)...
    mientras (!s.vacío()) {
        printf("%d ",s.arriba()); s.
        estallido();
    }
    printf("\n");}
```

Cola

Una estructura de datos que solo permite la inserción desde atrás (trasero), y solo permite la eliminación desde la cabecera (frente). Este comportamiento se llama Primero en entrar, primero en salir (FIFO), similar a una cola normal en el mundo real.

Operaciones importantes en la cola:

1. Encolar (C++ STL: push())

Agrega un nuevo elemento al final (parte trasera) de una cola.

2. Desencolar (STL de C++: pop())

Recupera y elimina el primer elemento de una cola que se encuentra al final (parte posterior) de la misma.

3. Mirar (C++ STL: top())

Recupera el primer elemento de una cola sin eliminarlo.

4. IsEmpty (C++ STL: empty())

Determina si una cola está vacía.

CONSEJOS: UTILIZACIÓN DE LA STL DE COLA DE C++

Implementar una cola estándar tampoco es difícil. De nuevo, ¿para qué complicarse? Simplemente use la biblioteca estándar de colas de C++.

```
#incluir <stdio.h>
# incluir <cola>

// Utilice esto para evitar especificar "std::" en todas partes. usando el
espacio de nombres std;

void main() {
    // Simplemente haz esto, escribe queue<el tipo que desees, //
    en este caso, integer> y el nombre de la cola. cola<int>q;

    // intenta insertar 7 enteros diferentes, no ordenados q.empujar(3);
    q.empujar(1); q.empujar(2);
    q.empujar(7); q.empujar(6); q.empujar(5);
    q.empujar(4);

    // El artículo que se inserta primero saldrá primero // Orden Primero en
    Entrar, Primero en Salir (FIFO)...
    mientras (!q.vacío()) {
        // ¡Observe que esto no es "top()" !!! printf("%d ",q.
        frente()); q.estallido();
    }
    printf("\n");}
```

CAPÍTULO 5 TÉCNICAS DE ENTRADA/SALIDA

En todos los concursos de programación, o más específicamente, en todos los programas útiles, necesitas leer datos de entrada y procesarlos. Sin embargo, los datos de entrada pueden ser tan desagradables como sea posible, y esto puede ser muy problemático analizarlo.[2]

Si has dedicado demasiado tiempo a programar cómo analizar la entrada de forma eficiente y utilizas C/C++ como lenguaje de programación, este consejo es para ti. Veamos un ejemplo:

Cómo leer la siguiente entrada:

```
1 2 3 1 2 3 1 2
```

La forma más rápida de hacerlo es:

```
#incluir <stdio.h>
entero N;
void main() {
    mientras(scanf("%d",&N==1)){
        proceso N... }
}
```

Hay N líneas, cada línea siempre comienza con el carácter '0' seguido de '.', luego un número desconocido de dígitos x, finalmente la línea siempre termina con tres puntos "...".

```
norte
0.xxxx...
```

La forma más rápida de hacerlo es:

```
#incluir <stdio.h>
caracteres dígitos[100];

void main() {
    scanf("%d",&N);
    para (i=0; i<N; i++) {
        scanf("0.%[0-9]...",&digits); // ¿Sorprendido? printf("Los
        dígitos son 0.%.s\n",digits);
    }
}
```

Este es el truco que muchos programadores de C/C++ desconocen. ¿Por qué decimos C++ cuando scanf/printf son rutinas estándar de C? Esto se debe a que muchos programadores de C++

"Se obligan" a usar cin/cout todo el tiempo, sin darse cuenta de que scanf/printf todavía se pueden usar dentro de todos los programas de C++.

El dominio de las rutinas de entrada/salida de los lenguajes de programación te será de gran ayuda en los concursos de programación.

Uso avanzado de printf() y scanf()

Para quienes hayan olvidado el uso avanzado de printf() y scanf(), recuerden los siguientes ejemplos:

```
scanf("%[ABCDEFGHJKLMNOPQRSTUVWXYZ]", &line); //line es una cadena
```

Esta función scanf() solo acepta letras mayúsculas como entrada para line y cualquier otro carácter que no sea A..Z finaliza la cadena. De manera similar, la siguiente función scanf() se comportará como gets():

```
scanf("%[^\\n]", line); //line es una cadena
```

Aprende los caracteres de terminación predeterminados de scanf(). Intenta leer todas las funciones avanzadas de scanf() y printf(). Esto te será útil a largo plazo.

Usando una nueva línea con scanf()

Si el contenido de un archivo (input.txt) es

```
abecedario
definición
```

Y el siguiente programa se ejecuta para tomar la entrada del archivo:

```
char input[100], ch;
void main(void)
{
    freopen("input.txt", "rb", stdin);
    scanf("%s", &input);
    scanf("%c", &ch);
}
```

A continuación se presenta una ligera modificación al código:

```
char input[100],ch;
void main(void)
{
    freopen("input.txt","rb",stdin);
    scanf("%s\n",&input);
    scanf("%c",&ch);
}
```

¿Cuál será su valor ahora? El valor de ch será '\n' para el primer código y 'd' para el segundo código.

¡Tenga cuidado al usar gets() y scanf() juntos!

También debes tener cuidado al usar gets() y scanf() en el mismo programa. Pruébalo con el siguiente escenario. El código es:

```
scanf("%s\n",&dummy);
obtiene(nombre);
```

Y el archivo de entrada es:

```
ABCDEF
bbbbbbXXX
```

¿Qué valor obtienes para el nombre? "XXX" o "bbbbbbXXX" (Aquí, "b" significa espacio en blanco).

Programas de entrada múltiple

Los "programas de entrada múltiple" son una invención del juez en línea. El juez en línea suele utilizar los problemas y datos que se presentaron por primera vez en concursos presenciales. Muchas soluciones a problemas presentados en concursos presenciales toman un único conjunto de datos, proporcionan el resultado correspondiente y finalizan. Esto no implica que los jueces proporcionen solo un conjunto de datos. De hecho, los jueces proporcionan varios archivos como entrada, uno tras otro, y comparan los archivos de salida correspondientes con la salida del juez. Sin embargo, el juez en línea de Valladolid proporciona solo un archivo como entrada. Inserta todas las entradas del juez en un único archivo y, al principio de este, escribe cuántos conjuntos de entradas hay. Este número coincide con el número de archivos de entrada que utilizaron los jueces del concurso. Ahora, una línea en blanco separa cada conjunto de datos. Por lo tanto, la estructura del archivo de entrada para el programa de entrada múltiple queda de la siguiente manera:

```
Entero N //que denota el número de conjuntos de entrada
- - línea en blanco---
Conjunto de entrada 1 //Como se describe en el enunciado del problema
- - línea en blanco---

Conjunto de entrada 2 //Como se describe en el enunciado del problema
- - línea en blanco---
Conjunto de entrada 3 //Como se describe en el enunciado del problema
- - línea en blanco---
.
.
.
- - línea en blanco---
Conjunto de entrada n //Como se describe en el enunciado del problema
- - fin del archivo--
```

Tenga en cuenta que no debe haber ningún espacio en blanco después del último conjunto de datos. La estructura del archivo de salida para un programa de entrada múltiple queda de la siguiente manera:

```
Salida para el conjunto 1 //Como se describe en el enunciado del problema
- - Línea en blanco---
Salida para el conjunto 2 //Como se describe en el enunciado del problema
- - Línea en blanco---
Salida para el conjunto 3 //Como se describe en el enunciado del problema
- - Línea en blanco---
.
.
.
- - línea en blanco---
Salida para el conjunto n //Como se describe en el enunciado del problema
- - fin del archivo--
```

El sistema de evaluación en línea de la USU no cuenta con programas de entrada múltiple como el de Valladolid. Prefiere recibir varios archivos como entrada y establece un límite de tiempo para cada conjunto de datos.

Problemas de los programas de entrada múltiple

Hay ciertos aspectos que debes considerar de manera diferente para programas con múltiples entradas. Aunque la especificación de entrada indique que la entrada finaliza con el fin de archivo (EOF), cada conjunto de datos termina con una línea en blanco, excepto el último, que finaliza con el fin de archivo. Además, ten cuidado con la inicialización de las variables. Si no se inicializan correctamente, tu programa podría funcionar con un solo conjunto de datos, pero generar una salida correcta para varios conjuntos. Todas las variables globales se inicializan a cero.^[6]

CAPÍTULO 6 MÉTODO DE FUERZA BRUTA

Esta es la técnica más básica para resolver problemas. Aprovechando que la computadora es realmente muy rápida.

La búsqueda exhaustiva utiliza el método de fuerza bruta, directo y de prueba y error para encontrar la respuesta. Este método debería ser casi siempre el primer algoritmo o solución que consideres. Si funciona dentro de las limitaciones de tiempo y espacio, úsalo: es fácil de programar y, por lo general, fácil de depurar. Esto significa que tendrás más tiempo para trabajar en los problemas más difíciles, donde la fuerza bruta no es lo suficientemente rápida.

Lámparas de fiesta

Se te proporcionan N lámparas y cuatro interruptores. El primer interruptor enciende y apaga todas las lámparas, el segundo las pares, el tercero las impares, y el último interruptor enciende y apaga las lámparas 1, 4, 7, 10,...

Dado el número de lámparas, N , el número de pulsaciones de botón realizadas (hasta 10.000) y el estado de algunas de las lámparas (por ejemplo, la lámpara 7 está apagada), indique todos los estados posibles en los que podrían encontrarse las lámparas.

Ingenuamente, por cada vez que se presiona un botón, hay que probar 4 posibilidades, para un total de 4^{10000} (aproximadamente 10^{6020}), lo que significa que no hay forma de realizar una búsqueda completa (este algoritmo en particular explotaría la recursión).

Al observar que el orden en que se presionan los botones no importa, este número se reduce a aproximadamente 10000^4 (aproximadamente 10^{16}), todavía demasiado grande para buscarlo por completo (pero ciertamente más cerca por un factor de más de 10^{6000}).

Sin embargo, pulsar un botón dos veces es lo mismo que no pulsarlo ninguna vez, así que lo único que hay que comprobar es si cada botón se pulsa 0 o 1 vez. Eso son solo $2^4 = 16$ posibilidades, un número de iteraciones que sin duda se pueden resolver dentro del límite de tiempo.

Los relojes

Un grupo de nueve relojes ocupa una cuadrícula de 3×3 ; cada uno marca las 12:00, las 3:00, las 6:00 o las 9:00. Tu objetivo es hacer que todos marquen las 12:00. Desafortunadamente, solo puedes manipular los relojes mediante nueve tipos de movimientos diferentes, cada uno de los cuales gira un subconjunto de los relojes 90 grados en el sentido de las agujas del reloj. Encuentra la secuencia más corta de movimientos que devuelva todos los relojes a las 12:00.

La solución más obvia sería una recursiva, que comprueba si existe una solución de 1 movimiento, 2 movimientos, etc., hasta encontrarla. Esto requeriría un tiempo de 9^k , donde k es el número de movimientos. Dado que k puede ser bastante grande, este método no se ejecutará con tiempos de ejecución razonables.

Tenga en cuenta que el orden de los movimientos no importa. Esto reduce el tiempo a k^9 , lo cual no supone una mejora suficiente.

Sin embargo, dado que realizar cada movimiento cuatro veces es lo mismo que no hacerlo ninguna, sabemos que ningún movimiento se repetirá más de tres veces. Por lo tanto, solo hay 49 posibilidades, lo que equivale a 262 072, lo cual, considerando la regla general de tiempo de ejecución de más de 10 000 000 de operaciones por segundo, debería funcionar a tiempo. La solución de fuerza bruta, teniendo en cuenta este conocimiento, es perfectamente adecuada.

Es hermoso, ¿verdad? Si tienes este tipo de capacidad de razonamiento, muchos problemas aparentemente difíciles pueden resolverse finalmente mediante la fuerza bruta.

Recursión

La recursión es genial. Muchos algoritmos necesitan implementarse recursivamente. Un algoritmo popular de fuerza bruta combinatoria, el retroceso, generalmente se implementa de forma recursiva. Tras destacar su importancia, estudiémoslo.

La recursión es una función, procedimiento o método que se llama a sí mismo repetidamente con un rango de argumentos más reducido (dividiendo un problema en problemas más simples) o con argumentos diferentes (es inútil usar la recursión con los mismos argumentos). Se repite hasta llegar a un caso base, que es tan simple o incluso más simple que el anterior, que se puede resolver fácilmente, o hasta que se cumple una condición de salida.

Una recursión debe detenerse (de hecho, todo el programa debe terminar). Para ello, debe alcanzarse un caso base o condición final válida. Una codificación incorrecta provocará un error de desbordamiento de pila.

Puedes determinar si una función es recursiva o no examinando su código fuente. Si una función o procedimiento se llama a sí mismo varias veces, es de tipo recursivo.

Cuando se llama a un método/función/procedimiento:

- La llamada está suspendida,
- estado de la persona que llama guardado,
- Nuevo espacio asignado para variables del nuevo método.

La recursión es una técnica potente y elegante que permite resolver un problema resolviendo otro más pequeño del mismo tipo. Muchos problemas en informática implican recursión, y algunos son inherentemente recursivos.

Si un problema puede resolverse de ambas maneras (recursiva o iterativa), entonces es recomendable optar por la versión iterativa, ya que es más rápida y consume menos memoria. Ejemplos: factorial, secuencia de Fibonacci, etc.

Sin embargo, también existen problemas que solo pueden resolverse de forma recursiva, o que resultan más eficientes mediante métodos recursivos, o cuyas soluciones iterativas son difíciles de conceptualizar. Ejemplos: la Torre de Hanoi, la búsqueda (DFS, BFS), etc.

Tipos de recursión

Hay 2 tipos de recursión, Lineal recursión y ramificación múltiple (Árbol) recursión.

1. La recursión lineal es una recursión cuyo orden de crecimiento es lineal (no ramificado). Un ejemplo de recursión lineal es el factorial, definido por $fac(n) = n * fac(n-1)$.

2. La recursión en árbol se ramifica en más de un nodo en cada paso, creciendo muy rápidamente. Nos proporciona formas flexibles de resolver problemas lógicamente complejos. Se puede usar para realizar una búsqueda exhaustiva (para que la computadora realice el método de prueba y error). Este tipo de recursión tiene un crecimiento cuadrático, cúbico o de mayor orden, por lo que no es adecuado para resolver problemas de gran tamaño. Se limita a problemas pequeños. Ejemplos: resolver la Torre de Hanoi, búsquedas (DFS, BFS), números de Fibonacci, etc.

Algunos compiladores pueden hacer que cierto tipo de recursión sea iterativa. Un ejemplo es la eliminación de la recursión de cola. La recursión de cola es un tipo de recursión en la que la llamada recursiva es el último comando en esa función/procedimiento.

Consejos sobre recursión

1. La recursión es muy similar a la inducción matemática.
2. Primero verás cómo puedes resolver el caso base, para $n=0$ o para $n=1$.
3. Luego, usted asume que sabe cómo resolver el problema de tamaño $n-1$ y busca una manera de obtener la solución para el problema de tamaño n a partir de la solución de tamaño $n-1$.

Al construir una solución recursiva, tenga en cuenta las siguientes preguntas:

1. ¿Cómo se puede definir el problema en términos de un problema más pequeño del mismo tipo?
2. ¿Cómo disminuye el tamaño del problema con cada llamada recursiva?

3. ¿Qué ejemplo del problema puede servir como caso base?

4. A medida que disminuye el tamaño del problema, ¿se alcanzará este caso base?

Ejemplo de una recursión muy estándar, Factorial (en Java):

```
factorial estático entero(int n) {  
    si (n==0)  
        devolver 1;  
    demás  
        devolver n*factorial(n-1);  
}
```

Divide y vencerás

Esta técnica utiliza la analogía de que "lo pequeño es más simple". Los algoritmos de divide y vencerás intentan simplificar los problemas dividiéndolos en subproblemas que se pueden resolver más fácilmente que el problema principal, para luego combinar los resultados y obtener una solución completa.

Algoritmos de divide y vencerás: Ordenación rápida, Ordenación por fusión, Búsqueda binaria.

La estrategia "Dividir y vencerás" consta de 3 pasos principales:

1. Dividir los datos en partes
2. Encontrar subsoluciones para cada una de las partes de forma recursiva (normalmente).
3. Construye la respuesta final al problema original a partir de las subsoluciones.

```
DC(P) {  
    si pequeño(P) entonces  
        devolver Solve(P);  
    demás {  
        Dividir P en instancias más pequeñas P1,...,Pk, k>1; Aplicar DC a  
        cada uno de estos subproblemas; devolver  
        combine(DC(P1),...,DC(Pk));  
    }  
}
```

La búsqueda binaria no sigue el patrón de la versión completa de Divide y vencerás, ya que nunca combina las soluciones de los subproblemas. Sin embargo, usemos este ejemplo para ilustrar la capacidad de esta técnica.

La búsqueda binaria te ayudará a encontrar un elemento en un array. La versión más simple consiste en recorrer el array elemento por elemento (búsqueda secuencial). La búsqueda se detiene cuando se encuentra el elemento (éxito) o cuando se llega al final del array (fracaso). Esta es la forma más simple y la menos eficiente.

Para encontrar un elemento en un array. Sin embargo, normalmente terminarás usando este método porque no todos los arrays están ordenados; primero necesitas un algoritmo de ordenación para ordenarlos.

El algoritmo de búsqueda binaria utiliza el método de "divide y vencerás" para reducir el espacio de búsqueda y se detiene cuando encuentra el elemento o cuando el espacio de búsqueda está vacío.

Para utilizar la búsqueda binaria, es requisito previo que el array sea un array ordenado. El ejemplo más común de array ordenado para la búsqueda binaria es la búsqueda de un elemento en un diccionario.

La eficiencia de la búsqueda binaria es $O(\log n)$. Este algoritmo se denominó "binario" porque divide el espacio de búsqueda en dos partes más pequeñas (binarias).

Optimización de su código fuente

Tu código debe ser rápido. Si no puede ser "rápido", al menos debe ser "lo suficientemente rápido". Si el límite de tiempo es de 1 minuto y tu programa actual se ejecuta en 1 minuto y 10 segundos, quizás te convenga hacer algunos ajustes en lugar de reescribir todo el código.

Generación vs. filtrado

Los programas que generan muchas respuestas posibles y luego eligen las correctas (por ejemplo, un programa para resolver el problema de las ocho reinas) son filtros. Los que dan con la respuesta correcta sin errores iniciales son generadores. Generalmente, los filtros son más fáciles (y rápidos) de programar y su ejecución es más lenta. Calcula si un filtro es suficiente o si necesitas intentar crear un generador.

Precomputación / Precálculo

A veces resulta útil generar tablas u otras estructuras de datos que permitan la búsqueda más rápida posible de un resultado. Esto se denomina preprocesamiento (donde se intercambia espacio por tiempo). Se pueden compilar los datos precalculados en un programa, calcularlos al inicio del programa o simplemente almacenar los resultados a medida que se calculan. Por ejemplo, un programa que debe convertir letras de mayúsculas a minúsculas cuando están en mayúsculas puede realizar una búsqueda en tabla muy rápida que no requiere condicionales. Los problemas de concursos suelen utilizar números primos; muchas veces es práctico generar una larga lista de primos para usarla en otras partes del programa.

Descomposición

Si bien hay menos de 20 algoritmos básicos utilizados en los problemas de concurso, el desafío de los problemas de combinación que requieren una combinación de dos algoritmos para su solución es

Resulta desalentador. Intenta separar las pistas de las distintas partes del problema para poder combinar un algoritmo con un bucle o con otro y resolver diferentes partes del problema de forma independiente. Ten en cuenta que, a veces, puedes usar el mismo algoritmo dos veces en partes diferentes (¡e independientes!) de tus datos para mejorar significativamente el tiempo de ejecución.

Simetrías

Muchos problemas presentan simetrías (por ejemplo, la distancia entre dos puntos suele ser la misma independientemente de la dirección en que se recorra el conjunto). Las simetrías pueden ser de dos, cuatro, ocho direcciones, etc. Intente aprovechar las simetrías para reducir el tiempo de ejecución.

Por ejemplo, con simetría de cuatro vías, se resuelve solo una cuarta parte del problema y luego se escriben las cuatro soluciones que comparten simetría con la única respuesta (preste atención a las soluciones auto-simétricas, que solo deben aparecer una o dos veces, por supuesto).

Resolver hacia adelante versus hacia atrás

Sorprendentemente, muchos problemas de concursos funcionan mucho mejor si se resuelven al revés que si se utiliza un ataque frontal. Presta atención al procesamiento de datos en orden inverso o a la creación de un ataque que examine los datos en un orden o de una forma distinta a la obvia.

CAPÍTULO 7 MATEMÁTICAS

Conversión de números base

El sistema decimal es el más familiar para nosotros, mientras que las computadoras están muy familiarizadas con los números binarios (y también con los octales y hexadecimales). La capacidad de convertir entre sistemas de bases es importante.

Para ello, consulta el libro de matemáticas.[2]

PON A PRUEBA TUS CONOCIMIENTOS BÁSICOS SOBRE CONVERSIONES

Resuelve problemas de UVA relacionados con la conversión de bases:

[343 - ¿Qué base es esta?](#) 353 -
[Las bases están cargadas](#) 389 -
[Básicamente hablando](#)

Gran Mod

El módulo (resto) es una operación aritmética importante y casi todos los lenguajes de programación ofrecen esta operación básica. Sin embargo, la operación de módulo básica no es suficiente si nos interesa encontrar el módulo de un número entero muy grande, lo cual será difícil y propenso a desbordamientos. Afortunadamente, aquí tenemos una buena fórmula:

$$(A*B*C) \bmod N == ((A \bmod N) * (B \bmod N) * (C \bmod N)) \bmod N.$$

Para convencerte de que esto funciona, mira el siguiente ejemplo:

$(7*11*23) \bmod 5$ es 1; veamos los cálculos paso a paso:

$$\begin{aligned} ((7 \bmod 5) * (11 \bmod 5) * (23 \bmod 5)) \bmod 5 &= ((2 * 1 * 3) \bmod 5) \\ &= (6 \bmod 5) = 1 \end{aligned}$$

Esta fórmula se utiliza en varios algoritmos, como en la prueba de Fermat Little para pruebas de primalidad:

$R = B^A \bmod M$ (B es un muy muy grande número). Al usar la fórmula, podemos obtener la respuesta correcta sin temor a desbordamiento.

Así es como se calcula rápidamente (utilizando el enfoque de Divide y Vencerás; consulte la exponenciación a continuación para obtener más detalles):

```

largo bigmod(long b,long p,long m) {
    si (p == 0)
        devolver 1;
    si no, si (p%2 == 0)
        return square(bigmod(b,p/2,m)) % m; // square(x) = x * x else
    devolver ((b % m) * bigmod(b,p-1,m)) % m;
}

```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE BIG MOD

Resuelve problemas de UVa relacionados con Big Mod:

[374 - Gran Mod](#)

[10229 - Fibonacci modular - más Fibonacci](#)

Entero grande

Hace mucho tiempo, un entero de 16 bits era suficiente: $[-2^{15}a\ 2^{15-1}] \sim [-32.768\ a\ 32.767]$.

Cuando las máquinas de 32 bits se popularizan, los enteros de 32 bits se vuelven necesarios. Este tipo de datos puede contener un rango de $[-2^{31}a\ 2^{31-1}] \sim [2.147.483.648\ a\ 2.147.483.647]$. Cualquier número entero con 9 dígitos o menos se puede calcular de forma segura con este tipo de datos. Si necesita usar el décimo dígito, tenga cuidado con el desbordamiento.

Pero el hombre es muy codicioso, se crean enteros de 64 bits, conocidos por los programadores como tipo de dato long long o tipo de dato int64. Tiene un rango impresionante que puede cubrir enteros de 18 dígitos completamente, más el dígito 19 parcialmente $[-2^{63-1}\ a\ 2^{63-1}] \sim [9,223,372,036,854,775,808\ a\ 9,223,372,036,854,775,807]$. Prácticamente, este tipo de datos es seguro para calcular la mayoría de los problemas aritméticos estándar, excepto algunos problemas.

Nota: para aquellos que no lo saben, en C, largo largo n se lee usando: `scanf("%lld",&n)`; y largo sin signo largo n se lee usando: `scanf("%llu",&n)`; **Para datos de 64 bits:** `typedef unsigned long long int64; int64 test_data=64000000000LL`

Ahora, la criptografía RSA necesita números de 256 bits o más. Esto es necesario, los humanos siempre queremos más seguridad, ¿verdad? Sin embargo, algunos creadores de problemas en concursos de programación también siguen esta tendencia. Un problema simple como encontrar el factorial n-ésimo se volverá muy, muy difícil una vez que los valores superen el tipo de datos entero de 64 bits. Sí, long double puede almacenar números muy grandes, pero en realidad solo almacena los primeros dígitos y un exponente, long double esno preciso.

Implementa tu propio aritmética de precisión arbitraria El algoritmo estándar de lápiz y bolígrafo que se enseña en la escuela primaria será su herramienta para implementar operaciones aritméticas básicas: suma, resta, multiplicación y división. Más sofisticado

Existen algoritmos para aumentar la velocidad de la multiplicación y la división. Las cosas se están complicando mucho.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE NÚMEROS ENTEROS GRANDES
Resuelve problemas de UVa relacionados con números enteros grandes: 485 - Triángulo de la muerte de Pascal 495 - Fibonacci Freeze - más Fibonacci 10007 - Cuenta los árboles - más fórmula catalana 10183 - ¿Cuántas mentiras? - más Fibonacci 10219 - ¡Encuentra los caminos! - más fórmula catalana 10220 - ¡Me encantan los números grandes! - más fórmula catalana 10303 - ¿Cuántos árboles? - más fórmula catalana 10334 - Rayo a través de las gafas - más Fibonacci 10519 - ¡¡Realmente extraño!! 10579 - Números de Fibonacci - más Fibonacci

Número de Carmichael

Un número de Carmichael es un número que no es primo pero tiene al menos 3 factores primos. Puedes calcularlos usando un generador de números primos y un algoritmo de factorización prima.

Los primeros 15 números de Carmichael son:

561,1105,1729,2465,2821,6601,8911,10585,15841,29341,41041,46657,52633,62745,63973

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LOS NÚMEROS DE CARMICHAEL
Resuelve problemas de la UVa relacionados con el número de Carmichael: 10006 - Número de Carmichael

Fórmula catalana

El número de árboles binarios distintos se puede generalizar como la fórmula de Catalan, denotada como $C(n)$.

$$C(n) = \frac{2n}{n+1} C_{n-1}$$

Si se le piden valores de varios $C(n)$, puede ser una buena opción calcular los valores de abajo hacia arriba.

Dado que $C(n+1)$ puede expresarse en $C(n)$, de la siguiente manera:

Catalán(n) =

$$\frac{2n!}{n! * (n+1)}$$

Catalán(n+1) =

$$\frac{2^{*(n+1)}}{(n+1)! * ((n+1)+1)} = (n+1)!$$

$$\frac{(2n+2) * (2n+1) * 2n!}{n! * (n+1) * n! * (n+2)} = (n+1)$$

$$\frac{(2n+2) * (2n+1) * 2n!}{(n+2) * n! * n! * (n+1)} = (n+1)$$

$$\frac{(2n+2) * (2n+1)}{(n+2)} * \text{Catalán}(n) (n+1) *$$

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LAS FÓRMULAS CATALANAS

Resuelve problemas de la UVa relacionados con la fórmula catalana:

10007 - Cuenta los árboles - * n! -> número de árboles + sus permutaciones

10303 - ¿Cuántos árboles?

Combinaciones de conteo - $C(N,K)$

$C(N,K)$ indica de cuántas maneras se pueden tomar N elementos de K en K . Esto puede suponer un gran desafío cuando N y/o K son muy grandes.

La combinación de (N,K) se define como:

$$\frac{n!}{(n-k)! * k!}$$

Para su información, el valor exacto de $100!$ es:

```
93.326.215.443.944.152.681.699.238.856.266.700.490.715.968.264.381.621.46
8.592.963.895.217.599.993.229.915.608.941.463.976.156.518.286.253.697.920
, 827.223.758.251.185.210.916.864.000.000.000.000.000.000.000.000
```

Entonces, ¿cómo calcular los valores de $C(N,K)$ cuando N y/o K son grandes, pero se garantiza que el resultado cabe en un entero de 32 bits?

Divide por el máximo común divisor antes de multiplicar (código de ejemplo en C)

```
mcd largo(a largo, b largo) {
    Si (a % b == 0) devuelve b; de lo contrario, devuelve el máximo común divisor (MCD) de b, a % b.
}
void Divbygcd(long& a,long& b) {
    largo g=mcd(a,b);
    a/=g;
    b/=g;
}
C largo(int n,int k){
    numerador largo=1,denominador=1,aMultiplicar,aDividir,i; si
    (k>n/2) k=nk; /* usar k menor */ para (i=k;i--){

        toMul=n-k+i;
        toDiv=i;
        Divbygcd(toMul,toDiv);          /* Siempre divide antes de multiplicar */
        Divbygcd(numerador,toDiv);

    Divbygcd(toMul,denominador);
        numerador*=aMul;
        denominador*=toDiv;
    }
    devolver numerador/denominador;
}
```

PON A PRUEBA TUS CONOCIMIENTOS DE $C(N,K)$

Resuelve problemas de UVa relacionados con combinaciones:

[369 - Combinaciones](#)

[530 - Enfrentamiento binomial](#)

Divisores y divisibilidad

Si d es un divisor de n , entonces n/d también lo es, pero d y n/d no pueden ser ambos mayores que $\sqrt{n}$.

2 es divisor de 6, por lo tanto, $6/2 = 3$ también es divisor de 6.

Sea $N = 25$. Por lo tanto, ningún divisor será mayor que la raíz cuadrada de $25 = 5$. (Los únicos divisores de 25 son 1 y 5).

Si tienes presente esa regla, puedes diseñar un algoritmo para encontrar un divisor mejor; eso es todo, ningún divisor de un número puede ser mayor que la raíz cuadrada de ese número en particular.

Si un número $N = a^i * b^j * \dots * c^k$ entonces N tiene $(i+1)*(j+1)*\dots*(k+1)$ divisores.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE DIVISIBILIDAD

Resuelve problemas de UVA relacionados con la divisibilidad:

294 - Divisores

Exponenciación

(Supongamos que la función `pow()` en `<math.h>` no existe...). A veces queremos realizar una exponenciación o elevar un número a la potencia n . Hay muchas maneras de hacerlo. El método estándar es la multiplicación estándar.

multiplicación estándar

```
exponente largo (base larga, potencia larga) {  
    largo i, resultado = 1;  
    para (i=0; i<potencia; i++) resultado *= base; devolver  
    resultado;}
```

Esto es lento, especialmente cuando la potencia es grande (tiempo $O(n)$). Es mejor usar la estrategia de divide y vencerás.

Divide y vencerás: exponenciación

```
long square(long n) { return n*n; } long  
fastexp(long base, long power) {  
    si (potencia == 0)  
        devolver 1;  
    else if (power%2 == 0)  
        devolver cuadrado(expectativa rápida(base, potencia/2)); de lo  
        contrario  
        devolver base * (fastexp(base, power-1));  
}
```

Usando la fórmula incorporada, $a^n = \exp(\log(a)*n)$ o `pow(a,n)`

```
#incluir <stdio.h>
# incluir <math.h>

void main() {
    printf("%lf\n",exp(log(8.0)*1/3.0));
    printf("%lf\n",pow(8.0,1/3.0));
}
```

PON A PRUEBA TUS CONOCIMIENTOS SOBRE EXPONENCIACIÓN

Resuelve problemas de UVA relacionados con la exponenciación:

113 - El poder de la criptografía

Factorial

El factorial se define naturalmente como $\text{Fac}(n) = n * \text{Fac}(n-1)$.

Ejemplo:

```
Fac(3) = 3 * Fac(2) Fac(3) = 3 * 2 *
Fac(1) Fac(3) = 3 * 2 * 1 * Fac(0)
Fac(3) = 3 * 2 * 1 * 1 Fac(3) = 6
```

Versión iterativa del factorial

```
factor largo(int n) {
    entero i, resultado = 1;
    para (i=0; i<n; i++) resultado *= i; devolver
    resultado;
}
```

Este es el mejor algoritmo para calcular el factorial. $O(n)$.

Fibonacci

La sucesión de Fibonacci fue inventada por Leonardo da Pisa (Fibonacci). Él la definió como una creciente población de conejos inmortales. La sucesión de Fibonacci es: 1, 1, 2, 3, 5, 8, 13, 21, ...

Relación de recurrencia para Fib(n):

```
Fib(0) = 0
Fib(1) = 1
Fib(n) = Fib(n-1) + Fib(n-2)
```

Versión iterativa de Fibonacci (utilizando programación dinámica)

```
int fib(int n) {
    entero a=1,b=1,i,c;
    para (i=3; i<=n; i++) {
        c = a+b;
        a = b;
        b = c;
    }
    devolver a;
}
```

Este algoritmo se ejecuta en tiempo lineal $O(n)$.

Método rápido para calcular rápidamente la secuencia de Fibonacci, utilizando la propiedad Matrix.

```
Divide_Conquer_Fib(n) {
    i = h = 1;
    j = k = 0;
    mientras(n > 0) {
        si(n%2 == 1) { // si n es impar
            t = j*h;
            j = i*h + j*k + t; i = i*k + t;
        }
        t = h*h;
        h = 2*k*h + t;
        k = k*k + t;
        n = (int) n/2;
    }devolverj;}
}
```

Esto se ejecuta en tiempo $O(\log n)$.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LA SECUENCIA DE FIBONACCI

Resuelve problemas de UVa relacionados con
Fibonacci: [495 - Fibonacci Freeze](#) [10183 - How](#)
[Many Fibs](#) [10229 - Modular Fibonacci](#) [10334 - Ray](#)
[Through Glasses](#) [10450 - World Cup Noise](#) [10579](#)
- Números de Fibonacci

Máximo Común Divisor (MCD)

Como su nombre indica, el algoritmo del Máximo Común Divisor (MCD) encuentra el máximo común divisor entre dos números a y b . El MCD es una técnica muy útil, por ejemplo, para reducir números racionales a su mínima expresión ($3/6 = 1/2$). El mejor algoritmo de MCD hasta la fecha es el algoritmo de Euclides. Euclides descubrió esta interesante igualdad matemática: $MCD(a,b) = MCD(b,(a \bmod b))$. Aquí se muestra la implementación más rápida del algoritmo de MCD.

```
int MCD(int a,int b) {  
    mientras (b > 0) {  
        a = a % b;  
        a ^= b;      b ^= a;      a ^= b;    }    devolver a;  
}
```

Mínimo común múltiplo (MCM)

El mínimo común múltiplo (MCD) y el máximo común divisor (MCD) son dos propiedades importantes de los números, y están interrelacionadas. Si bien el MCD se usa con más frecuencia que el MCM, también es útil aprender el MCM.

```
MCM (m,n) = (m * n) / MCD (m,n)  
  
MCM (3,2) = (3 * 2) / MCD (3,2) MCM (3,2) =  
6 / 1  
MCM (3,2) = 6
```

Aplicación del LCM -> para encontrar un tiempo de sincronización entre dos semáforos, si el semáforo A muestra el color verde cada 3 minutos y el semáforo B muestra el color verde cada 2 minutos, entonces cada 6 minutos, ambos semáforos mostrarán el color verde al mismo tiempo.

Expresiones matemáticas

Existen tres tipos de expresiones matemáticas/algebraicas: expresiones infijas, prefijas y postfijas.

Infijogramática de expresiones:

```
<infijo> = <identificador> | <infijo><operador><infijo>  
<operador> = + | - | * | / <identificador> = a | b | ... | z
```

Ejemplo de expresión infija: $(1 + 2) \Rightarrow 3$, el operador está en medio de dos operandos. La expresión infija es la normal forma en que los humanos calculan números.

Prefijo gramática de expresiones:

<prefijo> = <identificador> | <operador><prefijo><prefijo>

<operador> = + | - | * | / <identificador> = a | b | ... | z

Ejemplo de expresión prefija: $(+ 1 2) \Rightarrow (1 + 2) = 3$, donde el operador es el primer elemento. Un lenguaje de programación que utiliza esta expresión es Scheme.

La ventaja de la expresión prefija es que permite escribir: $(1 + 2 + 3 + 4)$ de esta manera $(+ 1 2 3 4)$, esto es más sencillo.

Sufijo gramática de expresiones:

<postfijo> = <identificador> | <postfijo><postfijo><operador>

<operador> = + | - | * | / <identificador> = a | b | ... | z

Ejemplo de expresión posfija: $(1 2 +) \Rightarrow (1 + 2) = 3$, el operador es el último elemento.

Calculadora de sufijos

Las computadoras realizan mejor los cálculos en notación posfija que en notación infija. Por lo tanto, al compilar su programa, el compilador convertirá sus expresiones en notación infija (la mayoría de los lenguajes de programación la utilizan) a notación posfija, la versión más manejable por las computadoras.

¿Por qué las computadoras prefieren Postfix a Infix?

Esto se debe a que las computadoras utilizan una estructura de datos de pila, y los cálculos en notación posfija se pueden realizar fácilmente utilizando una pila.

Conversión de infijo a postfijo

Para utilizar este cálculo posfijo tan eficiente, necesitamos convertir nuestra expresión infija a posfija. Así es como lo hacemos:

Ejemplo:

Declaración infija: $((4 - (1 + 2 * (6 / 3) - 5)))$. Esto es lo que hará el algoritmo, observe atentamente los pasos.

Expresión	Apilado (de abajo hacia arriba)	Expresión postfija
$((4 - (1 + 2 * (6 / 3) - 5)))$	(	
$((4 - (1 + 2 * (6 / 3) - 5)))$	((	
$((4 - (1 + 2 * (6 / 3) - 5)))$	((	4
$((4 - (1 + 2 * (6 / 3) - 5)))$	((-	4

$((4-(1+2*(6/3)-5)))$	$((-($	4
$((4-(1+2*(6/3)-5)))$	$((-($	41
$((4-(1+2*(6/3)-5)))$	$((-(+$	41
$((4-(1+2*(6/3)-5)))$	$((-(+$	412
$((4-(1+2*(6/3)-5)))$	$((-(+*$	412
$((4-(1+2*(6/3)-5)))$	$((-(+*($	412
$((4-(1+2*(6/3)-5)))$	$((-(+*($	4126
$((4-(1+2*(6/3)-5)))$	$((-(+*(/$	4126
$((4-(1+2*(6/3)-5)))$	$((-(+*(/$	41263
$((4-(1+2*(6/3)-5)))$	$((-(+*$	41263/
$((4-(1+2*(6/3)-5)))$	$((-($	41263/*+
$((4-(1+2*(6/3)-5)))$	$((-($	41263/*+5
$((4-(1+2*(6/3)-5)))$	$((-$	41263/*+5-
$((4-(1+2*(6/3)-5)))$	$($	41263/*+5--
$((4-(1+2*(6/3)-5)))$		41263/*+5--

PON A PRUEBA TUS CONOCIMIENTOS SOBRE INFijos Y POSTFijos

Resuelve problemas de UVa relacionados con la conversión posfija:

727 - Ecuación

Factores primos

Todos los números enteros se pueden expresar como un producto de números primos y estos primos se llaman **factores primos** de ese número.

Excepciones:

Para los números enteros negativos, se multiplica por -1 para que vuelvan a ser positivos. Para -1, 0 y 1, no hay factor primo. (Por definición...)

Método estándar

Genera de nuevo una lista de números primos y luego comprueba cuántos de ellos dividen a n. Esto es muy lento. Quizás puedas escribir un código muy eficiente usando este algoritmo, pero existe otro mucho más efectivo.

norte

$$\begin{array}{c} \backslash \\ 100 \\ / \backslash \\ \underline{2} \quad 50 \\ / \backslash \\ \underline{2} \quad 25 \\ / \backslash \\ \underline{5} \quad 5 \\ / \backslash \\ \underline{5} \quad \underline{1} \end{array}$$

Método creativo, utilizando la teoría de números.

1. No generes números primos, ni siquiera compruebes si son primos.
2. Siempre utilice **detenerse en la raíz cuadrada** técnica
3. Recuerda comprobar los factores primos repetidos; por ejemplo, 20 $\rightarrow 2 \cdot 2 \cdot 5$. No cuentes el 2 dos veces. Buscamos factores primos distintos (aunque en algunos problemas es necesario contar estos múltiplos).
4. Haga que su programa utilice un espacio de memoria constante, sin necesidad de arreglos.
5. Utilice con criterio la definición de factores primos; esta es la idea clave. Un número siempre se puede dividir en un factor primo y otro factor primo, o bien en otro número. Esto se conoce como árbol de factorización.

A partir de este árbol de factorización, podemos determinar las siguientes propiedades:

1. Si tomamos un factor primo (F) de cualquier número N, entonces N se hará cada vez más pequeño hasta que se convierta en 1, nos detenemos aquí.
2. Este N más pequeño puede ser un factor primo o puede ser otro número más pequeño, no nos importa, todo lo que tenemos que hacer es repetir este proceso hasta que $N=1$.

PON A PRUEBA TUS CONOCIMIENTOS SOBRE LOS FACTORES PRIMARIOS

Resuelve problemas de UVA relacionados con factores primos:

583 - Factores primos

Números primos

Los números primos son importantes en informática (por ejemplo, en criptografía) y encontrarlos en un intervalo determinado es una tarea ardua. Generalmente, buscamos números primos grandes (muy grandes), por lo que la búsqueda de mejores algoritmos para encontrar números primos nunca cesará.

1. Pruebas de prima estándar

```
int es_primo(int n) {
    para (int i=2; i<=(int) sqrt(n); i++) si (n%i == 0) devolver 0; devolver 1;
}

void main() {
    entero i, count=0;
    for (i=1; i<10000; i++) count += is_prime(i); printf("Total de %d
    primos\n", count);
}
```